

BOILER ROOM PROJEKT

Systemprogrammering och introduktion till C++

Kurs 3 - Systemutvecklare C/C++

PROJEKTINFORMATION

Projekt:

Local Energy Optimization Platform (LEOP)

Projektnamn:

Ta fram ett eget, unikt produktnamn

Beskrivning:

System för prognostisering och optimering av solel baserat på väder- och spotprisdata

Arbetstid:

4 timmar Boiler Room (måndag) + minst 8 timmar extra grupparbete per vecka

PROJEKTBESKRIVNING

Local Energy Optimization Platform (LEOP) är ett systemnära mjukvaruprojekt som syftar till att skapa en lokal plattform för prognostisering och optimering av solenergi. Systemet bygger vidare på tidigare erfarenheter från kurs 2 (klient-server-lösning för väderdata) och ska utvecklas till ett produktligt energisystem.

Projektet ska kännas som ett verkligt industriprojekt - ett lokalt körbart system utan molnberoende arkitektur som skulle kunna användas i smarta hem, energioptimering eller embedded-sammanhang. Genom att kombinera väderdata med spotprisdata från elnätet kan systemet beräkna optimala tider för elförbrukning, batteriladdning och försäljning av överskottsproduktion.

Projektet ska genomföras med agilt arbetssätt. Varje vecka bygger på föregående veckas workshops och introducerar successivt mer avancerade koncept.

SYSTEMKRAV

Följande krav är obligatoriska för projektet och måste uppfyllas för godkänt betyg.

Arkitektur

- Server implementerad i C
- Multi-threaded design med tydlig pipeline: fetch → parse → compute → cache
- Modulär arkitektur med tydliga ansvarsområden
- Minst en del av systemet körs i separat process
- Kommunikation via Unix domain sockets, pipes eller shared memory

Klientkrav

- Minst en CLI-klient implementerad
- C++-klient ska använda RAII och STL där lämpligt
- Klienten ska kunna hämta prognoser från servern
- Klienten ska kunna visa spotprisdata
- Klienten ska kunna visa beräknad energiplan

Robusthet och drift

- Felhantering för API-fel, timeout och ogiltig data
- Konfigurationsfiler för systeminställningar
- Möjlighet att starta och stoppa systemet kontrollerat
- Loggning av viktiga händelser

Prestanda

- Profilerings av beräkningssteget med gprof eller perf
- Identifiering av flaskhalsar i koden
- Dokumenterad optimering med före/efter-mätningar

FUNKTIONELLA KRAV

LEOP-systemet ska tillhandahålla följande funktionalitet för energioptimering.

Energidata och analys

- Hämta väderdata: solinstrålning, molnighet, temperatur
- Beräkna förväntad solcellsproduktion per 15 minuter
- Lagra prognoser lokalt med cache och TTL (Time-To-Live)
- Hämta spotprisdata per 15 minuter
- Matcha spotprisdata mot solprognos
- Beräkna optimala tider för elförbrukning, energilagring och försäljning

Optimering

Systemet ska generera en tidsbaserad energiplan (24-72 timmar) som visar:

- När systemet bör köpa el från nätet
- När egen solproduktion ska användas direkt
- När batteri ska laddas respektive urladdas
- När försäljning av överskottsproduktion är gynnsam

STRETCH GOALS - UTÖKAD FUNKTIONALITET

Följande funktionalitet är valfri och ger möjlighet för starkare grupper att utmana sig själva ytterligare.

Kategori	Stretch Goals
Avancerad IPC	• Shared memory med POSIX semaforer för cache • Message queues för asynkron kommunikation • Multiplexad I/O med select() eller poll()
Optimeringslogik	• Avancerade optimeringsalgoritmer för energiplanering • Maskininlärningsbaserad prognos • Batterimodellering med laddcykler och degradering
Skalbarhet	• Dynamisk trådpool baserad på systemlast • Connection pooling för databas/API-anslutningar • Distribuerad cache med invalidering
Prestanda	• SIMD-optimeringar för numeriska beräkningar • Cache-vänlig dataorganisering • Zero-copy I/O för stora dataöverföringar
Simulering	• Lastsimulering för hushållsförbrukning • Solpanelsmodellering med orientering och skuggning • Ekonomisk simulering med olika elpriser

LEVERABLER

Vid kursens slut (vecka 12) ska följande levereras:

Kod och system

- Komplet källkod i Git-repository
- Makefile eller build-script för kompilering
- Fungerande server och klient
- Konfigurationsfiler med dokumentation
- Automatiserade tester

Dokumentation

- README med installationsinstruktioner
- Arkitekturdokumentation med diagram
- API-dokumentation för server-klient-kommunikation
- Profileringsrapport med före/efter-mätningar
- Individuell skriftlig reflektion (per student)

Presentation

- Muntlig presentation (15-20 minuter)
- Live-demonstration av systemet
- Presentation av profileringsresultat och optimeringar
- Diskussion av designval och lärdomar

ARBETSSÄTT

Agilt arbetssätt

Projektet genomförs med agila metoder. Gruppen förväntas arbeta självorganiserande med följande praktiker:

- Sprintplanering i början av varje vecka
- Daily standups (kort avstämning inom gruppen)
- Backlog med prioriterade uppgifter
- Retrospektiv för att förbättra arbetssättet
- Git-baserat versionshantering med feature branches

Veckans struktur

Dag	Aktivitet
Måndag	Boiler Room (4 timmar) - Implementationsarbete i grupp
Tisdag	Checkpoint via Zoom - Visa framsteg och få vägledning
Onsdag	Workshop via Zoom - Genomgångar och code-alongs
Torsdag	Workshop via Zoom - Fortsatta övningar
Fredag	Undervisningsfri - Eget arbete

Förväntad arbetsinsats

Utöver schemalagd tid förväntas varje student lägga minst 8 timmar per vecka på projektarbetet.

Detta inkluderar (utöver studier av kursmaterial inför Boiler Room):

- Individuell kodning och testning
- Kodgranskning och feedback till gruppmedlemmar
- Dokumentationsarbete

REKOMMENDERAD VECKOPLANERING

Följande veckoplanering beskriver vad som ska implementeras varje vecka.

Observera att vecka 1 är en introduktionsvecka utan Boiler Room-implementation.

Från vecka 2 följer veckorna mönstret där måndagens Boiler Room implementerar det som lärdes ut föregående veckas workshops.

KURSVECKA 1: PROJEKTINTRODUKTION OCH PLANERING

Fokus: Introduktionsvecka - inget Boiler Room-arbete

Baskrav	• Läs igenom hela projektspecifikationen noggrant • Skapa en offert/lösningsförslag med arkitekturöversikt • Sätt upp Git-repository och grundläggande projektstruktur • Identifiera roller och ansvarsområden inom gruppen • Skapa en preliminär backlog för projektet
Kursmål	• Kursmål 12: Dokumentera design och överväganden
Checkpoint <i>(tisdag)</i>	Presentation av offert och lösningsförslag

KURSVECKA 2: SERVERARKITEKTUR OCH PROCESSHANTERING

Fokus: *Grundläggande serverstruktur med fork()*

Implementera grundstrukturen för LEOP-servern med processhantering. Servern ska kunna starta, skapa barnprocesser och hantera grundläggande livscykeloperationer.

Implementering	• Servers main-loop med fork() för att hantera anslutningar • Grundläggande signalhantering (SIGINT, SIGTERM) • Strukturerad felhantering vid processskapande • Loggningssystem för serverhändelser
Baskrav	• Server som kan startas och stoppas kontrollerat • Korrekt hantering av barnprocesser med wait()/waitpid() • Grundläggande loggning till fil eller stdout • Konfigurerbar serverport via kommandoradsargument
Stretch Goals <i>(valfritt)</i>	• Demonisering av servern (köra i bakgrunden) • PID-fil för processhantering • Graciös nedstängning med signaler
Kursmål	• Kursmål 1: Process- och trådhantering • Kursmål 7: Flertrådade program
Checkpoint <i>(tisdag)</i>	Demo av fungerande server som kan starta/stoppa och logga händelser

KURSVECKA 3: MULTI-THREADED PIPELINE

Fokus: Implementera flertrådad databehandling

Bygga den interna pipelinen för databehandling med trådar. Fokus på korrekt synkronisering mellan pipeline-stegen: fetch → parse → compute → cache.

Implementering	• Trådpool för parallell bearbetning • Producent-konsument-mönster mellan pipeline-steg • Mutex-skyddade delade resurser • Villkorsvariabler för koordinering
Baskrav	• Minst 3 trådar som hanterar olika pipeline-steg • Trådssäker kö mellan producent och konsument • Korrekt användning av pthread_mutex_lock/unlock • Inga race conditions vid testning
Stretch Goals (valfritt)	• Dynamisk trådpool som skalar efter last • Läs-skriv-lås för bättre parallellism på cache • Prestandamätning av pipeline-genomströmning
Kursmål	• Kursmål 1: Synkronisering • Kursmål 7: Effektiv synkronisering och resursdelning
Checkpoint (tisdag)	Demo av fungerande pipeline med synkroniserade trådar

KURSVECKA 4: IPC MED PIPES

Fokus: *Separera komponenter med interprocesskommunikation*

Implementera kommunikation mellan serverprocessen och externa komponenter via pipes. Minst en del av systemet ska köras i en separat process.

Implementering	• Datahämtare som separat process • Kommunikation via anonyma eller namngivna pipes • Protokoll för dataöverföring mellan processer • Felhantering vid pipe-kommunikation
Baskrav	• Minst en komponent som körs i separat process • Fungerande pipe-kommunikation mellan processer • Korrekt hantering av fildeskriptorer vid fork() • Hantering av stängda pipes och EOF
Stretch Goals <i>(valfritt)</i>	• Bidirektionell kommunikation med två pipes • Buffrad I/O för effektivare överföring • Timeout-hantering vid pipe-läsning
Kursmål	• Kursmål 2: IPC med pipes • Kursmål 8: IPC-lösningar för processkommunikation
Checkpoint <i>(tisdag)</i>	Demo av processkommunikation via pipes

KURSVECKA 5: UNIX SOCKETS OCH DELAT MINNE

Fokus: Utökad IPC-funktionalitet

Implementera Unix domain sockets för klient-server-kommunikation och utforska delat minne för högpresterande datadelning.

Implementering	• Unix domain socket-server för klientanslutningar • Enkel textbaserad protokoll för kommandon • Alternativt: POSIX shared memory för cache • Synkronisering med semaforer vid delat minne
Baskrav	• CLI-klient kan ansluta till servern via Unix socket • Server hanterar flera samtidiga klientanslutningar • Strukturerat kommunikationsprotokoll • Korrekt cleanup av socket-filer
Stretch Goals (valfritt)	• Shared memory-baserad cache med semaforer • Connection pooling för klienter • Asynkron I/O med select() eller poll()
Kursmål	• Kursmål 2: Sockets och delat minne • Kursmål 8: IPC-lösningar
Checkpoint (tisdag)	Demo av fungerande klient-server-kommunikation

KURSVECKA 6: C++-MIGRERING BÖRJAR

Fokus: Omstrukturera delar av systemet till C++

Börja migrera utvalda komponenter från C till C++. Fokus på att förstå skillnaderna och använda C++-idiom korrekt.

Implementering	• Identifiera komponenter lämpliga för C++-migrering • Konvertera datastrukturer till C++-klasser • Ersätta char* med std::string där lämpligt • Använda namespaces för organisering
Baskrav	• Minst en modul omskriven i C++ • Korrekt användning av references istället för pekare • Användning av const där lämpligt • Fungerande integration mellan C och C++-kod
Stretch Goals (valfritt)	• Funktionsöverlagring för flexibelt API • Default-argument för enklare funktionsanrop • Operator-överlagring för egna typer
Kursmål	• Kursmål 3: Skillnader mellan C och C++
Checkpoint (tisdag)	Demo av C++-migrerad komponent

KURSVECKA 7: C++ KLASSER OCH OBJEKTDESIGN

Fokus: Designa och implementera C++-klasser

Fortsätt C++-migreringen (vid behov) fokusera på objektorienterad design (C-moduler och/eller C++ klasser). Implementera moduler/klasser med korrekt livscykelhantering.

Implementering	• Moduler/klasser för konfiguration, cache och energiplan • Konstruktörer och destruktörer • Korrekt inkapsling med public/private • Kopieringskonstruktör och tilldelningsoperator
Baskrav	• Minst 3 väldesignade och städade moduler/klasser • Korrekt resurshantering i destruktörer • Tydlig separation mellan interface och implementation • Dokumenterade klassdiagram
Stretch Goals (valfritt)	• Statiska medlemmar för singleton-liknande konfiguration • Kedjning av metodanrop (fluent interface) • Kompositionsmönster för komplexa objekt
Kursmål	• Kursmål 4: C++-objektmodellen
Checkpoint (tisdag)	Demo av klassdesign och objektinteraktion

KURSVECKA 8: RAI OCH RESURSFÖRVALTNING

Fokus: Implementera RAI-mönster genomgående

Tillämpa RAI-principen för all resurshantering. Säkerställ att alla resurser (filer, lås, minne) hanteras korrekt.

Implementering	• RAI-wrappers för fildeskriptorer • RAI-wrappers för mutex-lås • Move-semantik för effektiv resursöverföring • Undantagshantering med try-catch
Baskrav	• Inga manuella delete eller free i C++-kod • RAI-klass för minst en systemresurs • Korrekt implementering av Rule of Three/Five • Grundläggande exception handling
Stretch Goals (valfritt)	• Move-semantik med rvalue references • noexcept-specifikation för kritiska funktioner • Exception-säkra konstruktörer
Kursmål	• Kursmål 4: RAI • Kursmål 9: RAI-implementation
Checkpoint (tisdag)	Demo av RAI-implementation och exception handling

KURSVECKA 9: STL-INTEGRATION

Fokus: Använda STL-komponenter effektivt

Integrera STL-komponenter i projektet. Ersätt manuella datastrukturer med `std::vector`, `std::map` och smarta pekare.

Implementering	• <code>std::vector</code> för dynamiska arrayer • <code>std::map/unordered_map</code> för cache • <code>std::unique_ptr</code> och <code>std::shared_ptr</code> • STL-algoritmer för databehandling
Baskrav	• Alla dynamiska arrayer ersätta med <code>std::vector</code> • Smarta pekare för all dynamisk allokering • Användning av minst 3 STL-algoritmer • Range-based for-loopar genomgående
Stretch Goals (valfritt)	• <code>std::optional</code> för nullable värden • <code>std::variant</code> för typsäkra unioner • Custom allocators för prestandakritisk kod
Kursmål	• Kursmål 5: STL-komponenter • Kursmål 9: STL-användning
Checkpoint (tisdag)	Demo av STL-integrerat system

KURSVECKA 10: PROFILERING OCH PRESTANDAANALYS

Fokus: Mäta och analysera systemets prestanda

Genomför systematisk profilering av systemet. Identifiera flaskhalsar och dokumentera prestandakarakteristik.

Implementering	• Profilering med gprof eller perf • Minnesanalys med Valgrind • Benchmarking med std::chrono • Prestandadokumentation
Baskrav	• Minst en profileringsrapport med gprof/perf • Minnesläckagetest med Valgrind • Dokumenterade hotspots i koden • Benchmarks för kritiska operationer
Stretch Goals (valfritt)	• Cache-analys med Cachegrind • Flamegraph-visualisering • Automatiserade prestandatester
Kursmål	• Kursmål 6: Profilering • Kursmål 10: Utföra profilering
Checkpoint (tisdag)	Presentation av profileringsresultat och identifierade flaskhalsar

KURSVECKA 11: OPTIMERING, FÄRDIGSTÄLLANDE & DOKUMENTATION

Fokus: *Optimera baserat på mätdata, färdigställ projektet och slutför dokumentation*

Implementera optimeringar baserade på profileringsresultat. Färdigställ teknisk dokumentation.

Implementering	• Optimering av identifierade flaskhalsar • Kompilatoroptimeringar (-O2, -O3) • Teknisk dokumentation med arkitekturdiagram • README och installationsinstruktioner
Baskrav	• Mätbar prestandaförbättring dokumenterad • Komplet teknisk dokumentation • Installationsguide och användardokumentation • Koddokumentation (kommentarer, Doxygen)
Stretch Goals (valfritt)	• Algoritmisk optimering av beräkningar • Cache-vänlig dataorganisering • SIMD-optimeringar för numeriska beräkningar
Kursmål	• Kursmål 6: Prestandaoptimering • Kursmål 11: Optimera kod • Kursmål 12: Dokumentera design
Checkpoint (tisdag)	Demo av optimeringar med före/efter-mätningar

KURSVECKA 12: EXAMINATION

Fokus: Slutpresentation och inlämning

Examinationsvecka. Projektet ska vara färdigställt och presentationsredo.

Baskrav (måste uppfyllas)	• Kompletta fungerande system • All dokumentation färdigställd • Förberedd presentation (15-20 minuter) • Individuell skriftlig reflektion
Kursmål	• Samtliga kursmål
Checkpoint (tisdag)	Slutpresentation och projekthinlämning

EXAMINATION

Projektet examineras genom tre moment som tillsammans täcker samtliga kursmål.

Moment	Beskrivning	Kursmål	Deadline
Skriftligt kunskapstest	Individuell prövning av teoretiska kunskaper om processer, trådar, IPC, C++ och profilering.	1-6	Kursvecka 12 (onsdag)
Skriftlig reflektion	Individuell redogörelse för egna lösningar, designval och lärdomar från projektet.	1-6	Kursvecka 12 (torsdag)
Projektinlämning	Gruppinlämning med kod, dokumentation och presentation. Varje students individuella insats bedöms.	7-12	Kursvecka 12 (torsdag)

Betygskriterier

Icke godkänt (IG): Samtliga kursmål är inte uppfyllda.

Godkänt (G): Samtliga kursmål är uppfyllda.

Väl godkänt (VG): Alla lärandemål är uppfyllda på en tillfredsställande nivå. Utöver detta uppvisar studenten:

- Hög precision att implementera flertrådade C- eller C++-program med effektiv synkronisering och korrekt resursdelning.
- Stor skicklighet att använda IPC-lösningar för processkommunikation i systemnära program.
- Hög precision att implementera C++-komponenter med RAII och grundläggande STL-användning.

Komplettering

För varje examinationsmoment har studenten rätt att delta på en ordinarie examination och en (1) omexamination. Kompletteringstillfälle erbjuds senast 3 veckor efter kursslut.

KURSMÅL - REFERENS

Följande kursmål ska uppfyllas genom projektarbetet och de teoretiska examinationsmomenten.

Kunskaper

1. Förklara hur operativsystem hanterar processer, trådar, synkronisering och minne.
2. Redogöra för interprocesskommunikation som pipes, sockets och delat minne.
3. Förklara skillnader mellan C och C++ i syntax, abstraktionsnivå och resursmodell.
4. Redogöra för C++-objektmodellen och RAII som metod för resursförvaltning.
5. Förklara hur STL-komponenter som vector, string och unique_ptr hanterar resurser.
6. Förklara hur profilering används för prestandaoptimering av systemnära kod.

Färdigheter

7. Implementera flertrådade C- eller C++-program med effektiv synkronisering och korrekt resursdelning.
8. Använda IPC-lösningar för processkommunikation i systemnära program.
9. Implementera C++-komponenter med RAII och grundläggande STL-användning.
10. Utföra profilering, tolka resultat och identifiera flaskhalsar.
11. Optimera kod baserat på mätdata och resursanalys.
12. Dokumentera design, minnesmodeller och prestandaöversväganden på ett strukturerat sätt.

Koppling mellan projektveckor och kursmål

Veckor	Kursmål	Fokusområde
Vecka 1-2	1, 7, 12	Processer, serverarkitektur
Vecka 3	1, 7	Trådar och synkronisering
Vecka 4-5	2, 8	IPC: pipes, sockets, delat minne
Vecka 6-7	3, 4	C++ migrering, klasser
Vecka 8	4, 9	RAII och resursförvaltning
Vecka 9	5, 9	STL-integration
Vecka 10-11	6, 10, 11, 12	Profilering, optimering, dokumentation
Vecka 12	Samtliga	Examination och presentation

TEKNISKA TIPS OCH RESURSER

Utvecklingsmiljö

- Använd Linux (Ubuntu rekommenderas) för utveckling
- Kompilera med gcc/g++ och flaggorna -Wall -Wextra -pthread
- Använd Valgrind för minnesläckagedetektering
- Använd GDB för debugging av flertrådade program
- Versionshantera med Git från dag ett

Kodkvalitet

- Följ en konsekvent kodstandard
- Dokumentera funktioner och klasser med kommentarer
- Skriv modular kod med tydliga gränssnitt
- Använd meningsfulla variabel- och funktionsnamn
- Undvik globala variabler - använd parametrar och structs

Vanliga fallgropar att undvika

- Race conditions vid delad data utan synkronisering
- Deadlocks vid inkonsekvent låsordning
- Minnesläckor vid glömd free/delete eller saknad RAII
- Zombieprocesser vid utebliven wait()
- Buffertöverskridning vid osäker stränghantering
- Otillräcklig felhantering vid systemanrop

Användbara kommandon

- gcc -Wall -Wextra -pthread -g -o server server.c (kompilering med debug)
- valgrind --leak-check=full ./server (minnesanalys)
- gprof ./server gmon.out > profile.txt (profilering)
- strace ./server (spåra systemanrop)
- ps aux | grep server (visa processer)
- lsof -i :8080 (visa öppna portar)

AVSLUTANDE ORD

LEOP-projektet är utformat för att ge er praktisk erfarenhet av systemnära programmering i en realistisk kontext. Genom att arbeta med processer, trådar, IPC och modern C++ kommer ni att utveckla färdigheter som är direkt tillämpbara i industrin.

Kom ihåg att:

- Planera noggrant innan ni börjar koda
- Dokumentera löpande - inte bara i slutet
- Testa tidigt och ofta
- Fråga om hjälp vid checkpoints
- Ha roligt och lär av varandra!

Lycka till med projektet!